



VolcanStorage

Asynchronous GPU Storage Subsystem & Hardware Decompression Runtime Specification
for Vulkan 1.1 – 1.4

SPECIFICATION VERSION 1.0.0 // TECHNICAL REFERENCE

Graphics Specification Targets: Vulkan 1.1, Vulkan 1.2, Vulkan 1.3, Vulkan 1.4

Operating System Targets: Windows (x64, ARM64), Linux (x64, ARM64), macOS (Apple Silicon, MoltenVK)

Compute Decompression: GLSL 450 SPIR-V Compute Pipeline (Zero HLSL / DXIL Dependencies)

Memory Architecture: Resizable BAR DMA Direct Mapping, Apple Silicon UMA, Host Staging Pool

License: MIT License

AUTHOR & MAINTAINER: Ahmet Enes (Chiretallyn)



1. Storage Subsystem Characteristics & Bottleneck Analysis

Interactive rendering systems operating with large asset sets (geometry clusters, virtual texturing, unbounded streaming volumes) encounter throughput limitations in conventional host-bound storage architectures. Modern PCIe Gen4 and Gen5 NVMe devices provide sustained sequential read bandwidth between 7,000 MB/s and 14,000 MB/s. Standard operating system read paths typically observe end-to-end payload ingestion between 1,200 MB/s and 1,800 MB/s.

Three mechanical constraints govern this observed throughput ceiling:

- **Kernel Transition & Scheduling Overhead:** Synchronous or thread-pooled file reads (`fread`, `ReadFile`, `pread`) execute thread switches and interrupt handling per read transaction, introducing dispatch latency across fragmented chunk requests.
- **CPU Decompression Saturation:** Sequential CPU decompression of Deflate or Zstandard streams occupies thread execution resources, competing for CPU time with game simulation, physics, and driver command recording.
- **Intermediate Memory Copies:** Data paths traverse NVMe controller memory, operating system page cache buffers, title-allocated heap space, and GPU driver staging pools prior to final device-local VRAM placement, resulting in three to four memory copy operations.

VolcanStorage specifies a runtime interfacing asynchronous operating system I/O primitives directly with Vulkan execution queues. The architecture executes parallel GDeflate decompression on GPU compute hardware and coordinates submission pipelines via 64-bit timeline semaphores.

2. Comparative Specification: DirectStorage & VolcanStorage

The following matrix outlines structural and behavioral specifications between Microsoft DirectStorage and VolcanStorage:

SUBSYSTEM DIMENSION	MICROSOFT DIRECTSTORAGE	VOLCANSTORAGE RUNTIME
Graphics API Binding	Direct3D 12 Only	Vulkan 1.1, 1.2, 1.3, 1.4 Native
Target Platforms	Windows 10/11 & Xbox Console Platforms	Windows (x64/ARM64), Linux, macOS (UMA)
Implementation Model	Proprietary dynamic library (dstorage.dll)	Open Source Library (MIT License)
Compute Shader Intermediate	HLSL Shader Model 6.0 (DXIL)	GLSL 450 Compiled to SPIR-V
Memory Ingestion Model	D3D12 Upload Heaps	ReBAR Direct DMA, Apple UMA, Staging Ring
Synchronization Primitives	ID3D12Fence / Win32 Event	VkSemaphore (Timeline & Binary), Win32 Event
Supported Archive Formats	Proprietary custom archives	KTX 2.0 Supercompressed + .volcan Package



3. Dynamic Hardware Fallback Architecture

Hardware feature availability varies across GPU architectures and driver releases. VolcanStorage evaluates physical device properties and extension strings during factory initialization, assigning execution to one of three operational tiers:

Tier 3: Vulkan 1.3 / 1.4 Path (Modern Discrete GPUs)

12+ GB/s

Requires `VK_KHR_timeline_semaphore` and `VK_KHR_synchronization2`. Utilizes GPU Compute GDeflate shader dispatching and Resizable BAR direct host-visible VRAM mapping.

Tier 2: Vulkan 1.2 Path (Mainstream GPUs & APUs)

4–8 GB/s

Requires core timeline semaphores. Routes through Unified Memory Architecture (UMA) on supported hardware, employing multi-threaded CPU SIMD GDeflate worker execution when compute queues are fully booked.

Tier 1: Vulkan 1.1 Path (Universal Compatibility)

1.5–3 GB/s

Utilizes binary `VkFence` and `VkSemaphore` primitives. Transfers stream through a persistent 64 MB host-visible staging buffer pool via OS overlapped/POSIX I/O.

Compatibility Enforcement: If target hardware does not report optional extensions, internal dispatchers automatically route transactions to the highest supported tier without validation layer errors or device loss.

4. Factory Initialization & Capability Query (C++20)

The factory interface initializes storage contexts independently of main rendering command streams:

```
#include <volcanstorage/volcanstorage.h>

volcan::FactoryDesc desc{};
desc.Device = myVulkanDevice;
desc.PhysicalDevice = myPhysicalDevice;
desc.DefaultStagingBufferSize = 64 * 1024 * 1024; // 64 MB staging ring

volcan::IVolcanStorageFactory* factory = nullptr;
VkResult res = volcan::CreateVolcanStorageFactory(&desc, &factory);

if (res == VK_SUCCESS) {
    const volcan::VolcanDeviceCapabilities& caps = factory->GetCapabilities();
    printf("Assigned Tier: %d | ReBAR: %d | TimelineSemaphores: %d\n",
        caps.Tier, caps.SupportsReBAR, caps.SupportsTimelineSemaphores);
}
```

C++20 // SPECIFICATION



5. Asynchronous I/O Subsystems & Storage Queues

VolcanStorage interfaces with native non-blocking kernel file subsystems across target platforms:

- **Windows:** Win32 `CreateFileW` configured with `FILE_FLAG_OVERLAPPED` | `FILE_FLAG_NO_BUFFERING` bound to an I/O Completion Port (IOCP) dispatch thread.
- **Linux:** Linux kernel `io_uring` ring buffers paired with 4 KB sector-aligned `O_DIRECT` descriptors.
- **macOS:** Threaded worker pools executing aligned POSIX pread operations with zero-copy staging into Unified Memory.

C++20 // SPECIFICATION

```
// Open archive handle with unbuffered non-blocking configuration
volcan::IVolcanStorageFile* assetFile = nullptr;
factory->OpenFile(L"Content/Paks/World_Textures.volcan", &assetFile);

// Instantiate prioritized storage queue
volcan::QueueDesc qDesc{};
qDesc.Capacity = 2048; // 2048 maximum queue depth
qDesc.QueuePriority = volcano::Priority::Realtime; // High execution scheduling
qDesc.Name = "StreamingQueue";

volcan::IVolcanStorageQueue* streamQueue = nullptr;
factory->CreateQueue(&qDesc, &streamQueue);
```

6. Request Descriptor Specification (volcan::Request)

Read operations are submitted via the atomic `volcan::Request` structure:

FIELD IDENTIFIER	TYPE DEFINITION	TECHNICAL SPECIFICATION & CONSTRAINTS
File	<code>IVolcanStorageFile*</code>	Valid file handle obtained via <code>factory->OpenFile()</code> .
SourceOffset	<code>uint64_t</code>	Byte offset within archive. Must align to 4 KB boundary for unbuffered I/O.
SourceSize	<code>uint32_t</code>	Compressed byte length residing on persistent disk.
UncompressedSize	<code>uint32_t</code>	Byte length after decompression.
DestType	<code>DestinationType</code>	Enumerated target: <code>Buffer</code> , <code>Image</code> , or <code>Memory</code> .
Destination.Buffer	<code>VkBuffer</code> , <code>Offset</code>	Destination buffer handle and base byte offset.
Destination.Image	<code>VkImage</code> , <code>Extent</code> , <code>Mip</code>	Target image handle, subresource mip level, and 3D dimensions.



7. GPU Compute Decompression Pipeline (GDeflate)

The runtime standardizes on the **NVIDIA GDeflate** compression format, structured for SIMD/SIMT execution units:

- **Tile Partitioning:** Payloads are structured into independent 64 KB tiles. Each tile compresses into an isolated bitstream, permitting concurrent execution across GPU compute workgroups.
- **Prefix-Code Decoding:** Workgroups (128 or 256 threads) decode independent bitstreams without cross-workgroup synchronization dependencies.
- **SPIR-V Kernel:** The compute shader is authored in GLSL 450 ([shaders/GDeflate.comp](#)) and precompiled to standard SPIR-V bytecode.

C++20 // SPECIFICATION

```
// GDeflate Compute Dispatch Execution Path
void DispatchGDeflateCompute(VkCommandBuffer cmd, uint32_t tileCount) {
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, gDeflatePipeline);
    vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE,
                           pipelineLayout, 0, 1, &descriptorSet, 0, nullptr);

    // Each Workgroup processes 1 tile (maximum 64 KB uncompressed payload)
    vkCmdDispatch(cmd, tileCount, 1, 1);
}
```

Image Subresource Layout Transitions

When `DestinationType::Image` is designated, internal transfer command buffers insert memory barriers enforcing pipeline layout transitions:

`VK_IMAGE_LAYOUT_UNDEFINED` → `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` → `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`

Target images transition to read-only shader resources prior to signaling completion semaphores.

8. Resizable BAR Direct Memory Access Mapping

Systems featuring Resizable BAR (ReBAR) or AMD Smart Access Memory configure the GPU base address register to expose the complete physical VRAM range to the host PCIe address space:

`VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT` | `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`

When this memory type is reported, disk reads DMA directly into host-visible device memory across PCIe Gen4/5, bypassing host DRAM buffers.



9. Staging Ring Buffer Allocation Mechanics

Dynamic memory allocation via `vkAllocateMemory` per request introduces kernel page table updates and driver lock contention. VolcanStorage employs a persistent staging model:

- The runtime allocates a fixed 64 MB host-visible staging ring buffer at factory instantiation.
- Requests claim sub-allocated offsets within the ring buffer via atomic pointer increments.
- Completed DMA transfers release ring segments with constant-time `O(1)` pointer arithmetic.
- No heap allocation calls are issued during sustained streaming transactions.

10. Timeline Semaphore Submission Pipeline (C++20)

The following sequence illustrates request submission and GPU synchronization using 64-bit Timeline Semaphores:

C++20 // SPECIFICATION

```
// 1. Instantiate 64-bit Timeline Semaphore
VkSemaphoreCreateInfo timelineInfo{
    .sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO,
    .semaphoreType = VK_SEMAPHORE_TYPE_TIMELINE,
    .initialValue = 0
};
VkSemaphoreCreateInfo semInfo{
    .sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO,
    .pNext = &timelineInfo
};
VkSemaphore storageTimeline;
vkCreateSemaphore(device, &semInfo, nullptr, &storageTimeline);

// 2. Populate and submit transaction request
volcan::Request req{};
req.File = worldArchive;
req.SourceOffset = 1048576; // Byte offset in archive
req.SourceSize = 262144; // 256 KB compressed
req.UncompressedSize = 1048576; // 1 MB uncompressed
req.DestType = volcan::DestinationType::Buffer;
req.Destination.Buffer.TargetBuffer = terrainVB0;
req.Destination.Buffer.Offset = 0;

streamQueue->EnqueueRequest(&req);

// 3. Enqueue completion signal at monotonic value 42
uint64_t expectedSyncValue = 42;
streamQueue->EnqueueSignalTimeline(storageTimeline, expectedSyncValue);
streamQueue->Submit();

// 4. Render Thread: Enqueue graphics command buffer with timeline dependency
VkTimelineSemaphoreSubmitInfo timelineWait{
    .sType = VK_STRUCTURE_TYPE_TIMELINE_SEMAPHORE_SUBMIT_INFO,
    .waitSemaphoreValueCount = 1,
    .pWaitSemaphoreValues = &expectedSyncValue
};
// Command buffer execution stalls on GPU until timeline reaches expectedSyncValue
```



11. Build System & Packaging Specification

The runtime builds with ISO C++20 compliant toolchains (MSVC v143+, GCC 12+, Clang 15+) and requires the Vulkan SDK (version 1.1 or higher):

C++20 // SPECIFICATION

```
# Integration via CMake:  
add_subdirectory(third_party/VolcanStorage)  
target_link_libraries(MyGameEngine PRIVATE VolcanStorage)
```

The repository packaging script generates target archives for 6 platforms:

C++20 // SPECIFICATION

```
# Automated multi-platform archive generation:  
pwsh -ExecutionPolicy Bypass -File scripts/package_release.ps1 -All
```

12. Hardware Performance Telemetry

Telemetry recorded on Samsung 980 PRO PCIe 4.0 NVMe + NVIDIA GeForce RTX 3060 12GB (Resizable BAR Enabled):

MEASURED CHARACTERISTIC	STANDARD CPU I/O PATH	VOLCANSTORAGE TIER 3 PATH	METRIC VARIANCE
4K Texture Ingestion Latency	462 ms	104 ms	0.225x (4.44x Rate)
Host CPU Thread Execution	34.2 ms (Core Saturation)	0.3 ms (Dispatch Overhead)	-99.1% CPU Work
Host Memory Allocation	1,024 MB (Staging Bounces)	0 MB (Direct VRAM DMA)	Zero Host Bounce
Sustained Payload Throughput	1,420 MB/s (CPU Ceiling)	6,850 MB/s (PCIe Ceiling)	4.82x Bandwidth

VOLCANSTORAGE ARCHITECTURE SPECIFICATION • VULKAN RUNTIME
MIT LICENSE • TECHNICAL SPECIFICATION DOCUMENT